Job Description:

Job Title: Data Engineer (PySpark)

About the Role: We are seeking a highly skilled Data Engineer with strong expertise in PySpark and the Cloudera Data Platform (CDP) to join our data engineering team. As a Data Engineer, you will be responsible for designing, developing, and maintaining scalable data pipelines that ensure high data quality and availability across the organization.

This role requires a strong background in big data ecosystems, cloud-native tools, and advanced data processing techniques. The ideal candidate should have hands-on experience with data ingestion, transformation, and optimization on the Cloudera Data Platform, along with a proven track record of implementing data engineering best practices.

You will work closely with other data engineers to build solutions that drive impactful business insights.

Responsibilities:

Data Pipeline Development: Design, develop, and maintain highly scalable and optimized ETL pipelines using PySpark on the Cloudera Data Platform, ensuring data integrity and accuracy.

Data Ingestion: Implement and manage data ingestion processes from various sources such as relational databases, APIs, and file systems into the data lake or data warehouse on CDP.

Data Transformation and Processing: Use PySpark to process, cleanse, and transform large datasets into meaningful formats that support analytical needs and business requirements.

Performance Optimization: Perform tuning of PySpark code and Cloudera components to optimize resource utilization and reduce ETL runtime.

Data Quality and Validation: Implement data quality checks, monitoring, and validation processes to ensure accuracy and reliability across the pipeline.

Automation and Orchestration: Automate workflows using tools such as Apache Oozie, Airflow, or similar orchestration tools within the Cloudera ecosystem.

Monitoring and Maintenance: Monitor pipeline performance and troubleshoot issues to ensure smooth operation.

General questionnaire

1. **Total data size handled is approximately 800 TB.**
   Ans: I want to be transparent about scale — our Kahramaa platform doesn't sit at 800 TB today, but let me tell you what it actually handles and how far it will scale.
   We have just under a million smart meters live right now — about 988,000 across Qatar — each sending a reading every 15 minutes. That generates roughly 95 million meter events every single day. Each reading carries the full payload — meter ID, timestamp, voltage, power consumption, flow rate depending on meter type, status flags — which comes to about 2.5 KB per event in JSON format. So daily raw data from IoT alone is about 237 GB. Add in the Oracle billing pulls and MySQL customer data every hour, and we're looking at roughly 250 GB of raw data flowing through the system every day.
   Over a month that's about 7.6 TB of raw data total. After we compress and store it in our Bronze layer, it comes down to about 1.5 TB. By the time it reaches our final Gold layer — where we've aggregated 15-minute readings up to daily totals — we're down to about 350 GB per month in the analytical store.
   In Synapse Analytics, which is where the business teams query the data, we currently hold about 230 GB across all our fact and dimension tables — that's roughly 1.1 billion rows. The biggest fact table, electricity consumption at daily grain, has about 360 million rows right now for two years of history.
   At full deployment — Kahramaa's plan is 2 million meters by 2027 — the daily raw data doubles to about 500 GB, and the Synapse pool grows to around 2 TB over 5 years. The architecture handles that because we've designed it to scale partitions and compute rather than redesign anything.
   So to directly answer the 800 TB question — the platform I've built processes at a scale that's smaller than that right now, but the design philosophy is identical to what you'd need at 800 TB. I'm happy to walk through what changes at that scale.

2. **Description of project along with roles and responsibilities.**
   Ans: Let me walk you through my current and most recent projects.
   At **Kahramaa — Qatar's national electricity and water utility** — I lead the Data Transformation and Self-Service Digitization initiative. The project is essentially modernizing how Qatar manages utility consumption data across the entire country. We have *988,000 smart meters* now live and reading every 30 minutes, feeding into a cloud-native Medallion Lakehouse I architected on Azure. My role is end-to-end: I own the ingestion pipelines from Oracle ERP, MySQL CRM, and IoT streams through Azure Event Hubs; I build the PySpark transformation logic in Databricks that processes raw meter readings into clean, analytics-ready data; and I deliver the Gold layer that powers Power BI dashboards for business teams. I'm also responsible for data governance — RBAC, encryption, ADLS access policies — and for pipeline monitoring through Azure Monitor and Databricks logs.
   Before that, at **Kyndryl building a solution for National Australia Bank**, I was Lead Data Engineer on a network change automation platform. We built a Lakehouse that ingested network device configurations from ServiceNow and Cisco controllers, then used PySpark and Databricks SQL to automatically identify which network changes were low-risk and could be executed without human intervention. We automated *40%+ of change requests with a 95%+ success rate*. My responsibility there covered everything from real-time telemetry streaming via Event Hubs to post-deployment validation pipelines and data governance with Azure Purview.
   At **Empower Retirement (part of JPMC)**, I was Senior Data and Automation Engineer. I migrated legacy financial data workflows from tools like QP Tool and DBC into Python-based Databricks pipelines, and built scalable ingestion from on-premise databases and legacy systems into ADLS Gen2. That was where I went deep on financial data quality — working with Great Expectations to build validation suites that ran at every layer boundary.

3. **Challenges faced while handling large-scale data computation.**
   Ans: Yes, a few challenges I faced were genuinely complex, especially in large-scale projects like Kahramaa and NAB.
   The first was the **small file problem from IoT streaming at Kahramaa**. With nearly a million meters sending data every 30 minutes, our streaming pipeline was generating **thousands of small files daily** in the Bronze layer. Over time, this slowed down queries significantly. I solved this by implementing a **weekly compaction (OPTIMIZE) process** and applying **Z-Ordering on meter_id and event_time**. This improved query performance drastically — from minutes to seconds.
   The second challenge was **data skew during aggregations**. Some zones had much higher meter density, so when we grouped data by zone, one partition became much larger than others. This caused certain tasks to run much longer. I fixed this by enabling **Adaptive Query Execution (AQE)** and handling skew using **salting techniques**, which helped distribute the data more evenly.
   The third challenge was **schema evolution**, especially in my NAB/Kyndryl project. Different network devices were sending data in slightly different JSON formats for the same fields. This caused inconsistencies in processing. I handled this by enabling **schema evolution (mergeSchema)** in the bronze layer and then creating a **standardized silver layer** that mapped all variations into a common structure.
   Overall, these challenges taught me how to handle **file management, skew, and schema variability at scale**, ensuring both performance and reliability in production systems.

4. **Details of failures encountered in production environments.**
   Ans: Three production incidents stand out from my experience.
   First, a **streaming pipeline stopped silently** — no errors, but no new data was coming in. When I checked Databricks, I found the **checkpoint was corrupted**, so the stream didn't know where to resume. I fixed it by resetting the checkpoint and restarting the job. After that, I added a **data freshness alert** so we get notified if data stops arriving.
   Second, we had **Oracle connection failures during peak hours**. Our pipeline was opening too many parallel connections, and the database couldn't handle it. I fixed it by **reducing parallelism and staggering pipeline runs**, and scheduling higher load during off-peak hours.
   Third, a **data quality issue where the pipeline succeeded but data was missing**. The output had fewer records due to an unintended filter in the code. I fixed the logic and reprocessed the data. Then I added a **reconciliation check** to compare source and target counts so the pipeline fails if data is missing.
   These incidents helped me focus on **monitoring, controlled parallelism, and data validation** to make pipelines more reliable.

5. **Data quality checks implemented.**
   Ans: I usually think about data quality in **layers**, meaning we validate data at multiple stages as it moves through the pipeline, not just at the end.
   First check — at ingestion:
   As soon as data lands, whether it's IoT meter readings or Oracle records, I validate the basics — **mandatory fields like meter ID, timestamp, and valid consumption values**. If anything is missing or invalid, negative, I don't drop it. Instead, I move it to a **quarantine zone (reject layer)** with a reason logged, so it can be reviewed later.
   Second check — during transformation (Bronze to Silver):
   Here I implement **reconciliation checks**. I compare **record counts between source and target**, and also validate **aggregates like total consumption**. If we extract 5 million records, we expect 5 million after processing. Any mismatch triggers a failure — this helps catch silent data loss.
   Third check — business rule validation:
   This is where domain logic comes in. For example, I flag **anomalies** like zero consumption during active hours or unusually high spikes compared to historical averages. These records are not rejected but **flagged for investigation**, which is important in real-world operations.
   Fourth check — referential integrity:
   I ensure every record is valid in context — for example, every meter reading must map to a valid **meter ID in the master registry**. If not, it indicates an upstream data issue.
   Finally, all these checks are **automated and monitored**, and I expose them through **dashboards and alerts** so the team can track data health daily without digging into logs.
   Overall, my approach ensures **schema validation, reconciliation, business validation, and referential integrity** — covering both technical and business data quality.

6. **Business insights derived from the data.**
   Ans: Before smart meters, Kahramaa only had **monthly consumption totals**, so visibility was very limited. With 30-minute interval data from nearly a million devices, we were able to unlock several important business insights.
   One key insight was **peak demand by area**. We could now clearly see which neighbourhoods consume the most electricity during peak hours, like 6 PM to 9 PM. This helped the operations team plan **grid capacity and substation upgrades proactively**, instead of relying on estimates.
   Another major use case was **water leak detection**. We identified patterns like continuous low consumption during late-night hours, which usually indicates a leak. We built daily reports to flag such cases, allowing the maintenance team to do **targeted inspections** instead of waiting for customer complaints.
   We also derived insights around **seasonal and cultural patterns**, especially during Ramadan. Consumption behavior changes significantly during that period, so using historical interval data, we helped the business **adjust energy generation schedules in advance**, which reduced operational costs.
   Finally, for **billing disputes**, customer service teams can now access detailed 30-minute readings. This allows them to clearly explain when high consumption occurred, leading to **faster and more transparent resolution** instead of relying on estimated bills.
   Overall, these insights helped Kahramaa move from **limited monthly visibility to real-time, data-driven decision making**, improving both operational efficiency and customer experience.

7. **Feature computation approaches used.**
   Ans: By features I understand you mean the derived values we calculate from the raw readings — the extra columns we compute that make the data useful for analysis, beyond just the raw consumption numbers.
   Rolling averages: For each meter, we calculate the average consumption over the past 30 days. This is updated daily. It's used for the anomaly detection — if today's reading is more than three times the 30-day average, the system flags it. Technically, this is done using a

sliding window calculation over the ordered time-series data — we look back 30 days from each reading's date and compute the average within that window.
Time-of-use classification: Qatar has different electricity tariffs for different times of day. So for every electricity reading, we add a label — either Peak (6 PM to 10 PM), Shoulder (6 AM to 6 PM), or Off-Peak (10 PM to 6 AM) — based on the reading's timestamp. This label is what Kahramaa uses to calculate the correct charge for each reading in the billing process.
Customer consumption tier: We classify every customer into one of four bands — Low, Medium, High, or Very High — based on their total consumption over the past 90 days. This gets recalculated daily. The business uses this for demand response programmes — for example, reaching out to Very High consumers to offer efficiency tips or incentive programmes.
All of these computed values are calculated at the point where we move data from the raw layer to the clean layer — in our Databricks processing environment using PySpark — and then stored alongside the raw data so downstream reports don't need to recalculate them.

8. **Interaction with business stakeholders or counterpart teams.**
   Ans: My interaction with business stakeholders was quite regular and collaborative, especially in projects like Kahramaa where data directly impacts operations and billing.
   I usually start by **understanding the business requirement in simple terms**, not just the technical ask. For example, when they asked for anomaly detection, I worked with them to define what actually counts as an anomaly — like unusual consumption patterns or leaks — instead of assuming it technically.
   I had frequent discussions with **operations teams, billing teams, and customer service teams**, because each of them used the data differently. For instance, operations focused on peak demand and outages, while billing teams cared about accuracy and tariff application.
   During development, I ensured **continuous feedback loops** — sharing sample outputs, validating logic, and making sure the data matched their expectations. This helped avoid rework later.
   I also made it a point to **translate technical outputs into business language**, so stakeholders could easily understand insights without needing to know the underlying data complexity.
   Overall, my approach was to act as a bridge between **technical implementation and business understanding**, ensuring that what we build is both correct and actually useful to the business.

9. **Compliance and security checks implemented to avoid breaches.**
   Ans: In my projects, especially in regulated environments like utilities and finance, I implemented security and compliance checks at multiple layers to avoid any data breaches.
   At the access level, I enforced **RBAC (Role-Based Access Control)** using **Azure AD and Unity Catalog**, ensuring users only have access to the data they need — following the **principle of least privilege**. For sensitive data, I implemented **column-level masking and row-level security**, so PII (personal identification information) fields are protected.
   At the network level, I used **private endpoints and VNet integration** to make sure data never travels over the public internet. All data is secured with **encryption at rest and in transit**.
   From a data governance perspective, I used tools like **Microsoft Purview** for **data classification, cataloging, and lineage tracking**, which helps in identifying sensitive data and maintaining audit trails.
   I also enabled **audit logging and monitoring**, so every access and change is tracked. Alerts are configured for any unusual access patterns.
   On the pipeline side, I ensured **secure credential management** using Key Vault — no secrets are hardcoded. Additionally, we validate data inputs to prevent corrupt or unexpected data from entering the system.
   Finally, for compliance (like GDPR-type requirements), I ensured **data retention policies, controlled access, and the ability to trace and delete data when required**.
   Overall, the approach is layered — **access control, encryption, governance, monitoring, and secure pipeline design** — to ensure end-to-end protection of data.

10. **Scheduling mechanisms used in pipelines.**
    Ans: We use different scheduling approaches depending on the type of data, because not everything should run on the same schedule.
    For **smart meter readings**, it's completely **continuous processing**. The data arrives every 30 minutes from devices, so we run a **streaming pipeline in Databricks using Structured Streaming** that listens to the message queue and processes data in real time. There's no schedule — it runs 24/7.
    For **Oracle billing data**, we use a **time-based incremental schedule**, typically **every hour**. The pipeline uses a watermark to pull only new or updated records, so we avoid full data loads and keep it efficient.
    For **data cleaning and transformation (Bronze to Silver)**, we run pipelines **every two hours**. This gives a balance — enough frequency for freshness, but also enough buffer time to handle retries if something fails.
    For **daily reporting (Gold layer)**, we schedule jobs **once a day at 2 AM**, before business hours, so it doesn't impact user queries and dashboards are ready in the morning.
    For **event-driven scenarios**, like **SharePoint document uploads**, we use **event-based triggers**. As soon as a file arrives, the pipeline is triggered automatically — no need to wait for a schedule.
    Finally, for **maintenance tasks** like compaction and cleanup (OPTIMIZE, VACUUM), we schedule them **weekly during low-usage periods**, typically weekends, to avoid impacting performance.
    So overall, we use a mix of **continuous streaming, time-based scheduling, event-driven triggers, and maintenance windows**, depending on the workload and business needs.

11. **Testing mechanisms implemented in the project.**
    Ans: In my projects, I implemented testing at multiple levels to ensure both **data correctness and pipeline reliability**.
    At the development level, I validate transformations using **sample datasets and unit-level checks**, especially in PySpark — verifying joins, aggregations, and business logic before deploying.
    During pipeline execution, I use **data validation checks** like **schema validation, null checks, and data type checks** to ensure incoming data is correct.
    One key mechanism I implemented is **reconciliation testing** — comparing **source vs target record counts and aggregates** (like total consumption). If there's any mismatch beyond a threshold, the pipeline fails. This helps catch silent data loss.
    I also implemented **deduplication checks** using unique keys and Delta MERGE to ensure no duplicate records are introduced during incremental loads.

For business validation, I verify **key metrics and aggregates** with stakeholders, especially for billing and reporting, to ensure outputs match business expectations.
At the orchestration level, I test **end-to-end pipeline runs**, including failure scenarios, retry logic, and dependency handling in ADF.
Finally, after deployment, I rely on **monitoring and alerting** as a continuous validation layer — ensuring any anomaly or failure is detected immediately.
Overall, my approach combines **unit testing, data validation, reconciliation, and end-to-end testing**, ensuring both technical and business-level correctness.

12. **Use of Sonar software for code quality and durability.**
    Ans: We use **SonarQube** to ensure code quality, maintainability, and long-term reliability of our data pipelines.
    Primarily, it helps us identify **code smells, bugs, and potential vulnerabilities** early in the development cycle. For example, in our PySpark and Python code, Sonar flags issues like unused variables, complex logic, or inefficient patterns that can affect readability and performance.
    It also enforces **coding standards and best practices** across the team. This is important in data engineering projects where multiple developers contribute — it keeps the code consistent and easier to maintain.
    Another key aspect is **technical debt tracking**. Sonar gives a clear view of how much effort is needed to improve the codebase, which helps us prioritize refactoring activities.
    We also use **quality gates**, where code cannot be merged unless it meets certain thresholds — like no critical issues, acceptable coverage, and minimal duplication. This ensures only clean, reliable code goes into production.
    Overall, Sonar helps us maintain **high-quality, readable, and stable code**, which directly improves the durability and reliability of our data pipelines over time.

13. **Test coverage maintained at 80 percent.**
    Ans: Test coverage means: out of all the lines of code in your program, how many of those lines are actually run when your tests execute? 80% means at least 8 out of every 10 lines are covered by at least one test.
    I treat 80% as a floor — the minimum acceptable — not a target. In the Kahramaa code, I aim to be above that by specifically making sure I test the cases that matter most, not just the straightforward ones.
    The easy 60% comes from testing the normal happy-path scenarios — valid data in, correct output out. The important extra 20% comes from testing the edge cases: what happens when a value is null? What happens when the amount is exactly zero — the boundary between valid and invalid? What happens when the database connection isn't available and the pipeline has to handle that gracefully? Those edge cases are where real production bugs hide.
    I measure coverage using a tool called pytest-cov, which generates a report after the tests run showing exactly which lines weren't covered. I check this before merging any code change. If coverage drops below 80%, the check fails and the code can't be merged until more tests are added.
    One thing I'd specifically mention — I'm careful about the difference between line coverage and branch coverage. Line coverage says "this line was executed." Branch coverage says "every possible path through this if-else was tested." For validation logic and business rules, I always check branch coverage because that's where the real risk is.

14. **Monthly data volume handled is approximately 500 TB.**
    Ans: Our monthly volume at Kahramaa comes to about 7.6 TB of raw data — 7.1 TB from the smart meter stream and about 500 GB from our batch sources like Oracle and MySQL. That's the honest number from our environment.

    Now, 500 TB a month at a bank like ENBD is a very different beast. In a banking environment you're capturing every transaction in real time, every risk calculation gets logged, audit trails for every data access, regulatory reports, market data feeds — all of it. A utility meter sends one reading every 15 minutes that's a couple of kilobytes. A bank transaction touches dozens of tables and triggers logging across multiple systems. So the same number of customers generates much more data in banking than in utilities.

    What I can say is that the engineering approach is the same at any scale. At 7.6 TB a month I already deal with all the problems that show up at 500 TB — how to only extract what's new rather than pulling everything every time, how to store it efficiently so it takes up far less space than the raw size, how to make sure queries against billions of rows stay fast, and how to keep pipelines running reliably even when individual jobs have problems. Those are the exact same problems at 500 TB, just with bigger machines and more parallelism.

    If we scale up the cluster and increase the number of parallel database connections, the monthly throughput scales linearly. There's no architectural redesign needed — just configuration tuning.

    If they push further — "but have you handled that scale?"
    Be honest: "Not at 500 TB/month, no. The largest raw volume I've directly managed is 7.6 TB/month at Kahramaa. I'm telling you the honest number because if I claimed 500 TB and you asked me what the daily event count was or what Synapse DWU I was running at, I wouldn't be able to give you a number I actually stand behind. What I can tell you is exactly how I'd get there — because the architecture is already built to scale."
15. **Handling issues across multiple jobs (e.g., issues in 10 jobs out of 100 jobs).**
    Ans: When handling issues across multiple jobs — for example, if 10 out of 100 jobs fail — I focus on **isolation, prioritization, and root cause analysis**, rather than treating everything the same.
    First, I quickly **identify patterns**. I check whether those 10 failures are related — for example, same source system, same dependency, or same time window. In my experience, many times multiple job failures trace back to a **single upstream issue**, like a source outage or schema change.
    Next, I ensure **failure isolation**. Each job is designed to be independent and idempotent, so failures in some jobs don’t impact others. The successful 90 jobs should proceed without any issue.
    Then I **prioritize based on business impact**. Not all jobs are equally critical — for example, billing or real-time pipelines get immediate attention, while lower-priority reporting jobs can wait.
    For resolution, I analyze logs from ADF and Databricks to identify whether it’s a **data issue, performance issue, or infrastructure issue**, and fix accordingly. If needed, I rerun only the failed jobs rather than the entire pipeline.

I also ensure **retry mechanisms and alerts** are in place so transient failures are handled automatically, and the team is notified only when intervention is needed.
Finally, I document the root cause and implement preventive fixes — like better validation, dependency checks, or throttling — to avoid similar failures in the future.
Overall, the approach is to **contain the failure, fix efficiently, and prevent recurrence**, without disrupting the entire system.

1. Data Validation / Data Quality

16. How do you validate that the records loaded into the target table are correct and complete? What checks do you perform?

Ans: Validating correctness and completeness is something I treat as a multi-layered process — not a single check. In my Medallion architecture at Kahramaa, I apply validation at every layer boundary. Let me walk you through exactly what I do.

The first thing I check after any load is row count reconciliation. I compare the count of records in the source extraction to the count of records written to the target. If I extracted 5 million records from Oracle ERP, I expect 5 million records in Bronze — accounting for any intentional deduplication. If the numbers don't match, the pipeline halts and alerts.

Second is financial sum reconciliation — for any table involving monetary or measured values, I compare the SUM(amount) or SUM(kwh_consumed) between source and target. A row count can match while individual values are wrong, so sum validation is the catch-all for value-level corruption.

Third is null checks on mandatory fields. Columns like customer_id, transaction_id, meter_id, and transaction_date must never be null in the target. I explicitly check COUNT(*) WHERE mandatory_col IS NULL = 0 before promoting any batch.

Fourth is primary key or uniqueness validation — every record in a fact table must have a unique business key. I validate that COUNT(DISTINCT key) equals COUNT(*) after the load.

Fifth is referential integrity — every foreign key in the fact table must have a matching record in the dimension table. For example, every meter_id in FactElectricityConsumption must exist in DimMeter. An orphaned key means a join failure downstream and wrong or missing rows in reports.

Sixth is date range sanity — MIN(transaction_date) and MAX(transaction_date) in the target must fall within the expected processing window. If my daily pipeline for 2024-01-15 contains records dated 2023-11-01, something went wrong in the watermark logic.

At Empower/JPMCI formalized all six of these as Great Expectations expectation suites. Each suite ran as a Databricks task that gated Silver→Gold promotion. Any failed expectation failed the task with a descriptive error — not a silent pass. The validation results were persisted to a Delta metadata table and visualized in a Power BI data quality dashboard.

17. Apart from record count reconciliation, what other validation techniques are used in ETL pipelines?

Ans: Great question because record count is actually one of the weakest validations by itself — it tells you nothing about the values inside the records. Here's what I go beyond it with.
Aggregate sum reconciliation — as I mentioned, comparing SUM of all numeric columns between source and target catches value-level corruption that a row count never would.
Statistical distribution checks — at **Kahramaa**, I validate that the average daily electricity consumption per meter stays within a historical range. If today's average is 10x the 30-day average, that signals either a data quality issue or a genuinely anomalous day that needs investigation. I use Great Expectations' expect_column_mean_to_be_between for this.
Schema validation — I validate that the schema of the target Delta table matches the documented contract before any downstream pipeline reads from it. Delta Lake enforces this at write time, but I also run an explicit schema comparison between the source extraction schema and the expected target schema to catch type drift at the source before it causes a downstream failure.
Duplicate detection — I check for duplicate records based on the business key. This is especially important for IoT data where the Siemens AMI system can retransmit readings on network recovery, giving us two records for the same meter at the same timestamp.
Value range validation — every business column has a physically or logically plausible (simply means **believable** or **could reasonably be true**) range. Water consumption can't be negative. A single meter can't show 1 million kWh in a day. I define these bounds explicitly and flag any record outside them as anomalous rather than silently ingesting it.
Allowed values / domain validation — categorical columns like transaction_status should only contain values from a known set (COMPLETED, PENDING, FAILED). Any value outside that set is a data quality problem — possibly a schema change in the source system that wasn't communicated.
Cross-table consistency checks — at the Gold layer, I validate that the total billing amount in FactBilling reconciles back to the sum of consumption in FactElectricityConsumption for the same customer and period. If they don't match within a tolerance, it means the billing calculation has a logic bug.

18. How do you perform record-level and column-level validation between source and target systems?

Ans: This is where you go from aggregate-level assurance to row-level truth. Let me walk through both layers.
For column-level validation, I perform a column-by-column profile comparison between source and target. For every column in the load, I compare: row count, distinct count, null count, min, max, and sum (for numerics). If any of these diverge between source and target beyond an acceptable tolerance, I flag that specific column rather than failing the entire pipeline — this makes triage much faster.

When I say I perform a **column-by-column profile comparison between source and target**, it means I validate that the data is not just moved, but also **accurately preserved across every column**.Instead of only checking total record counts, I compare each column individually between source and target using key metrics like:

- **Row count consistency**
- **Null count per column**
- **Minimum and maximum values**
- **Average or sum (for numeric fields like consumption)**
- **Distinct counts (to check duplicates or missing values)**
  For example, if I move meter data from Oracle to ADLS, I don't just check "5 million records moved." I also validate:
- Meter IDs match in count and uniqueness
- Consumption totals match exactly
- No unexpected nulls or missing values
- Data ranges are consistent
  This helps catch **silent issues**, like:
- Data truncation
- Type conversion problems
- Missing or duplicated records
  So overall, column-by-column profiling ensures **data integrity at a detailed level**, not just at a high level.

In PySpark at **Kahramaa**, I do this like so:

```
from pyspark.sql.functions import col, count, countDistinct, sum as s, min as mn, max as mx, isnull

def column_profile(df, col_name):
  return df.agg(
    count(col(col_name)).alias("row_count"),
    countDistinct(col(col_name)).alias("distinct_count"),
    count(isnull(col(col_name)).cast("int") * 1
        .when(isnull(col(col_name)), 1).otherwise(0)).alias("null_count"),
    mn(col(col_name)).alias("min_val"),
    mx(col(col_name)).alias("max_val")
  )

src_profile = column_profile(source_df, "amount")
tgt_profile = column_profile(target_df, "amount")
# Compare and assert
```

For record-level validation, I use a key-based anti-join pattern. After loading, I compare the set of primary keys in source against the set of keys in target.
For **record-level validation**, I use a **key-based anti-join pattern**.
What this means is — after loading data, I compare the **primary keys from the source** with the **primary keys in the target** to make sure no records are missing or extra.
Technically, I perform:

- A **left anti-join from source to target** → this shows records that exist in source but are missing in target
- A **left anti-join from target to source** → this shows any unexpected or duplicate records in target
  If both sides return zero records, it means the data is perfectly aligned at the record level.
  This approach is very effective because it catches **exact missing or extra records**, not just count mismatches. It's something I've used in production pipelines, especially for **billing and critical datasets**, where even a single missing record can cause issues.
  So overall, anti-join validation ensures **100% record-level accuracy between source and target**.

```
# Records in source but missing from target (load omissions)
missing_in_target = source_df.select("txn_id").subtract(target_df.select("txn_id"))

# Records in target but not in source (phantom records)
phantom_in_target = target_df.select("txn_id").subtract(source_df.select("txn_id"))

assert missing_in_target.count() == 0, f"Missing records: {missing_in_target.count()}"
assert phantom_in_target.count()  == 0, f"Phantom records: {phantom_in_target.count()}"
```

For value-level comparison on individual records, I join source and target on the primary key and then compare each column using a hash or direct comparison:

```
# Join source and target on key, then compare columns
joined = (source_df.alias("s")
  .join(target_df.alias("t"), "txn_id", "inner"))

mismatches = joined.filter(
  (col("s.amount")   != col("t.amount"))   |
  (col("s.customer_id") != col("t.customer_id")) |
  (col("s.status")    != col("t.status"))
)

if mismatches.count() > 0:
  mismatches.show(20)
  raise ValueError(f"Column-level mismatches found: {mismatches.count()} records")
```

19. **If the source system sends a column as DATE and the target stores it as TIMESTAMP, how do you validate correctness?**

Ans: This is a very practical scenario and one I've handled in production — Oracle sends transaction_date as a DATE type, and our target Delta table stores it as TIMESTAMP for compatibility with streaming data and time-travel queries.

The validation has two parts: confirming the **date value is preserved**, and confirming the **time component defaults correctly to midnight**.

```
from pyspark.sql.functions import col, cast, hour, minute, second, to_date, count, when

# — Validation 1: Date value preserved after cast —
date_mismatch = (source_df.alias("s")
  .join(target_df.alias("t"), "txn_id")
  .filter(
    col("s.transaction_date") !=
    to_date(col("t.transaction_date"))   # cast TIMESTAMP back to DATE for compare
  )
)
assert date_mismatch.count() == 0, "Date values changed during DATE→TIMESTAMP cast"

# — Validation 2: Time component is exactly 00:00:00 —
bad_timestamps = target_df.filter(
  (hour("transaction_date")   != 0) |
  (minute("transaction_date") != 0) |
  (second("transaction_date") != 0)
)
assert bad_timestamps.count() == 0, "Non-midnight time components found in TIMESTAMP column"

# — Validation 3: No nulls introduced by the cast —
null_after_cast = target_df.filter(col("transaction_date").isNull()).count()
assert null_after_cast == 0, f"Cast introduced {null_after_cast} null timestamps"
```

In SQL terms, the equivalent check in Spark SQL is:

```
SELECT COUNT(*) AS bad_count
FROM target_transactions
WHERE
  HOUR(transaction_date)   != 0
  OR MINUTE(transaction_date) != 0
  OR SECOND(transaction_date) != 0;
-- Expected result: 0
```

Watch out for timezone shifts If Oracle stores dates in a local timezone (e.g., Asia/Qatar, UTC+3) and your Spark cluster runs in UTC, a midnight date in Oracle becomes 21:00:00 the previous day in UTC. Always configure your ADF or JDBC connection with an explicit sessionInitStatement=ALTER SESSION SET TIME_ZONE='UTC' to prevent timezone conversion bugs. This is a silent data corruption that row count checks will never catch.

20. **If the timestamp column automatically populates 00:00:00 for minutes and seconds, how do you verify correctness?**

Ans: When the time component defaults to 00:00:00 — which is expected when a DATE is cast to TIMESTAMP — the correctness verification is actually straightforward: you're confirming the default happened correctly and consistently for every record, not just most of them.

```
from pyspark.sql.functions import hour, minute, second, col, count, when, lit

# Verify EVERY record has 00:00:00 — not just the majority
time_audit = target_df.agg(
  count(when(hour("transaction_date") == 0, 1)).alias("hour_zero_count"),
  count(when(minute("transaction_date") == 0, 1)).alias("minute_zero_count"),
  count(when(second("transaction_date") == 0, 1)).alias("second_zero_count"),
  count("*").alias("total_count")
)
time_audit.show()
# All three counts must equal total_count
# If hour_zero_count < total_count → some records have non-zero hour → BUG

# Single-number assertion
non_midnight = target_df.filter(
  (hour("transaction_date") != 0) |
  (minute("transaction_date") != 0) |
  (second("transaction_date") != 0)
).count()

print(f"Records with non-midnight time: {non_midnight}")
# Pass condition: non_midnight == 0
```

There's also a useful spot check pattern: display a sample and visually confirm the pattern:

```
# Quick eyeball check — confirm pattern is consistent
target_df.select(
  "txn_id",
  "transaction_date",
```

```
    hour("transaction_date").alias("hh"),
    minute("transaction_date").alias("mm"),
    second("transaction_date").alias("ss")
).show(5, truncate=False)
# Expected:
# | T001 | 2024-01-15 00:00:00 | 0 | 0 | 0 |
# | T002 | 2024-01-16 00:00:00 | 0 | 0 | 0 |
```

Why this matters beyond the obvious If even one record has a non-zero time component, it means that record came from a different source code path — possibly a streaming event that carries a real timestamp, mixed into what should be a pure batch DATE load. Catching this immediately tells you there's a pipeline design bug, not just a data issue.

21. **If record counts match but column values are incorrect, how do you detect the issue?**

Ans: This is precisely why I say row count is a necessary but not sufficient validation. A matching count means you have the right number of records — it says nothing about whether the values inside them are correct. Here's my approach to detecting value-level corruption.

Sum reconciliation across all numeric columns. I compare SUM(amount) in source against SUM(amount) in target. If counts match but sum diverges, it means some records have incorrect values — possibly from a type truncation (a DECIMAL(18,4) loaded into a DECIMAL(10,2) silently drops precision), a unit conversion error, or a transformation bug.

Hash-based record comparison (Counts match But sum does NOT match). For exact value matching, I compute a row hash in both source and target using all columns, then compare the hash sets:

- Count check → tells if records exist
- Sum check → tells if numeric values are correct
- **Hash check → tells if the entire row is exactly the same**

```
from pyspark.sql.functions import md5, concat_ws, col

# Compute row hash for each record in source and target
src_hashed = source_df.withColumn(
    "row_hash",
    md5(concat_ws("|",
        col("txn_id").cast("string"),
        col("customer_id"),
        col("amount").cast("string"),
        col("transaction_date").cast("string"),
        col("status")
    ))
).select("txn_id", "row_hash")

tgt_hashed = target_df.withColumn(
    "row_hash",
    md5(concat_ws("|",
        col("txn_id").cast("string"),
        col("customer_id"),
        col("amount").cast("string"),
        col("transaction_date").cast("string"),
        col("status")
    ))
).select("txn_id", "row_hash")

# Find records where hash doesn't match (same key, different values)
mismatch = (src_hashed.alias("s")
    .join(tgt_hashed.alias("t"), "txn_id")
    .filter(col("s.row_hash") != col("t.row_hash"))
)
print(f"Value mismatches: {mismatch.count()}")

# Drill into which columns differ on the mismatched records
mismatched_keys = mismatch.select("txn_id")
drill = (source_df.join(mismatched_keys, "txn_id").alias("s")
        .join(target_df.join(mismatched_keys, "txn_id").alias("t"), "txn_id"))

drill.select(
    col("txn_id"),
    col("s.amount").alias("src_amount"),
    col("t.amount").alias("tgt_amount"),
    col("s.status").alias("src_status"),
    col("t.status").alias("tgt_status")
).show(20)
```

This pattern gives you exactly which records have wrong values and exactly which columns differ — making root-cause analysis fast and precise.

22. **How do you identify bad or inconsistent data during ETL processing?**

Ans: I think about bad data in three categories and handle each differently: **structurally bad** (nulls, wrong types, schema violations), **logically bad** (values outside business rules), and **contextually bad** (values that are individually valid but inconsistent with other data).

Structurally bad data is caught at the Bronze layer. Delta Lake schema enforcement rejects records with wrong types. I add explicit null checks for mandatory columns. Anything rejected goes to the quarantine Delta table with an error code — not lost, but not in the main pipeline either.

Logically bad data is caught at the Bronze → Silver transition. I tag each record with any rule violations as a computed column array, then split valid and invalid records:

```
from pyspark.sql.functions import col, when, lit, array, concat_ws, size, filter as arr_filter

flagged = raw_df.withColumn("error_flags", array(
  when(col("amount") <= 0,                    lit("NEGATIVE_OR_ZERO_AMOUNT")),
  when(col("customer_id").isNull(),           lit("NULL_CUSTOMER_ID")),
  when(col("txn_id").isNull(),                lit("NULL_TXN_ID")),
  when(col("transaction_date") > col("current_date"),  lit("FUTURE_DATE")),
  when(~col("status").isin("COMPLETED","PENDING","FAILED"),lit("INVALID_STATUS"))
)).withColumn(
  "active_errors",
  arr_filter("error_flags", lambda x: x.isNotNull())
).withColumn("is_valid", size(col("active_errors")) == 0)

valid_df     = flagged.filter(col("is_valid"))
quarantine_df = flagged.filter(~col("is_valid"))

# Route: valid → Silver, quarantine → dead-letter table
valid_df.drop("error_flags", "active_errors", "is_valid") \
  .write.format("delta").mode("append").save("/mnt/silver/transactions")

quarantine_df.write.format("delta").mode("append") \
  .save("/mnt/quarantine/transactions")
```

Contextually bad data — things like a customer showing up in transactions but absent from the customer master, or a meter reading from a device that was decommissioned six months ago — requires cross-table validation using joins. I run these as post-load Silver quality checks and route any referential violations to a separate quality issue table for business review.

23. **Provide an example where column-level or record-level validation was performed.**

Ans: Let me give you a concrete production example from **Kahramaa**.

We were ingesting daily electricity meter readings from Oracle ERP into our Silver Delta table. One morning, our Power BI dashboard showed total daily consumption for zone 7 was *4x higher than the 30-day average*. The pipeline had completed with a green status and row count reconciliation passed. But something was wrong.

I ran a column-level reconciliation between the Bronze and Silver layer for that day:

```
# Compare Bronze vs Silver for 2024-01-15
date_filter = "reading_date = '2024-01-15'"

bronze_profile = (spark.sql(f"""
  SELECT zone_id,
       COUNT(*) as row_count,
       SUM(kwh_consumed) as total_kwh,
       AVG(kwh_consumed) as avg_kwh,
       MAX(kwh_consumed) as max_kwh
  FROM bronze.electricity_readings
  WHERE {date_filter}
  GROUP BY zone_id
"""))

silver_profile = (spark.sql(f"""
  SELECT zone_id,
       COUNT(*) as row_count,
       SUM(kwh_consumed) as total_kwh,
       AVG(kwh_consumed) as avg_kwh,
       MAX(kwh_consumed) as max_kwh
  FROM silver.electricity_readings
  WHERE {date_filter}
  GROUP BY zone_id
"""))

bronze_profile.join(silver_profile, "zone_id", "full") \
  .withColumn("sum_delta", col("silver_total_kwh") - col("bronze_total_kwh")) \
  .filter(col("sum_delta") != 0) \
  .show()
```

The output showed that zone 7 in Silver had a sum *exactly 10x* higher than Bronze for the same date. Same row count, but the sum was wrong. I then drilled into the records with the highest values and found the issue: a unit conversion step in our Silver transformation was multiplying kWh by 10 for a specific meter firmware version that was reporting in *tenths of a kWh* — a format change that came in a Siemens

firmware update without a data contract notification from the source team. The fix was a firmware version lookup in the transformation logic with the correct divisor per version. We quarantined the affected Silver records, reprocessed with the fix, and the numbers reconciled perfectly.

Lesson from this incident Row count reconciliation would never have caught this. Only the column-level sum reconciliation exposed it. This is why aggregate reconciliation on every numeric column is non-negotiable — not optional or "nice to have."

2. Hive / Data Engineering Operations

24. In daily work, do you primarily use Hive queries or PySpark?

Ans: In my current and recent roles, I primarily use PySpark and Spark SQL — that's been the dominant tool across Kahramaa, Kyndryl/NAB, and Empower/JPMC. My platform stack is Azure Databricks and Delta Lake, where Spark is the native engine. But I want to be clear about the relationship between these tools, because it's not an either/or.

For complex, multi-step transformations — window functions, deduplication logic, incremental merge operations, streaming pipelines — I write PySpark. It gives me full programmatic control, testability through unit tests, and access to the full Spark ecosystem including MLlib and Structured Streaming.

For ad-hoc analysis, quick profiling, and Gold layer aggregations that are essentially SQL in nature, I use Spark SQL from within Databricks notebooks — which is functionally Hive-compatible SQL running on Spark. So I'm writing HiveQL-style queries regularly, just through the SparkSession SQL interface rather than Hive CLI or Beeline directly.

Honest Bridge to ENBD's CDP Environment: In a Cloudera CDP environment like yours, the native query tool is HQL (Hive Query Language) running on Hive or Impala. My Spark SQL experience maps directly — the syntax is virtually identical, the underlying semantics are the same. The key difference is execution engine: my queries run on Spark, yours on Hive/YARN. I'm comfortable adapting to Beeline or HUE as the query interface. Where I'd spend the most time learning is CDP-specific optimizations like Hive LLAP (Low Latency Analytical Processing) which is Cloudera's equivalent to Databricks' Delta cache.

25. How is Hive typically used in workflows (validation, transformation, analysis)?

Ans: Based on my understanding of Hive in a CDP ecosystem — both from my Spark/HiveContext integration experience and from what I've learned about the CDP architecture — Hive plays three distinct roles in a data engineering workflow.

As the metadata and catalog layer (always-on role). The Hive Metastore is the central catalog — it stores table definitions, partition metadata, schema information, and storage location mappings for all tables whether they're queried through Hive, Spark, or Impala. Even when you're not writing HQL queries, Hive Metastore is working in the background. When I read a Hive table from PySpark using spark.table("schema.table_name"), I'm hitting the Hive Metastore to resolve the table location and schema.

For batch ETL and transformation. In CDP workflows, Hive is typically used for heavy batch transformations — INSERT OVERWRITE, CTAS (Create Table As Select), and partition-based data movement operations. For example, loading a daily partition into a fact table:

```
INSERT OVERWRITE TABLE enbd.fact_transactions
PARTITION (txn_year=2024, txn_month=1)
SELECT
  txn_id, customer_id, amount,
  TO_DATE(transaction_date)  AS transaction_date,
  branch_id, status
FROM enbd.staging_transactions
WHERE txn_year = 2024 AND txn_month = 1;
```

For validation and profiling queries. HQL is used for quick row count checks, aggregate comparisons, and data quality validation queries between staging and target. These are often the same SQL checks I run in Spark SQL — COUNT, SUM, GROUP BY with HAVING — just executed through Beeline or HUE against the Hive engine.

For analysis and reporting. Hive or Impala (Cloudera's faster SQL-on-Hadoop engine) serves the analytical query layer, similar to how Databricks SQL serves that role in my Azure environment. Business users query Hive-registered tables through BI tools like Tableau or Power BI via JDBC/ODBC connections.

3. Hive Partition Management Scenario

26. If a Hive table partitioned by transaction_date loses partitions, how would you restore them?

Ans: When Hive partition metadata is lost — which means the Hive Metastore no longer knows about partitions that still physically exist in HDFS — there are three approaches depending on the scope and nature of the loss.

Approach 1 — MSCK(MetaStore Constancy Kheck) REPAIR TABLE (the standard recovery command). This is the go-to for external Hive tables. It scans HDFS for directories that match the partition scheme and registers any missing partitions back into the Metastore.

```
-- Full partition scan and repair
MSCK REPAIR TABLE enbd.transactions;

-- Verify the repair worked
SHOW PARTITIONS enbd.transactions;

-- Count check post-repair
SELECT COUNT(*) FROM enbd.transactions WHERE transaction_date = '2024-01-15';
```

MSCK REPAIR limitation: MSCK REPAIR does a full HDFS directory scan — on large tables with hundreds or thousands of partitions, this can take 10–20 minutes and puts load on the NameNode. For a table with one year of daily partitions (365 directories), it's manageable. For a table with 5 years of hourly partitions (43,800 directories), you need a targeted approach instead.

Approach 2 — ALTER TABLE ADD PARTITION (targeted recovery). When you know exactly which partitions are missing — for example, you ran a monitoring query and identified specific dates — you add them directly rather than scanning everything:

```
-- Add specific missing partitions manually
ALTER TABLE enbd.transactions ADD IF NOT EXISTS
  PARTITION (transaction_date='2024-01-14')
    LOCATION '/warehouse/enbd/transactions/transaction_date=2024-01-14'
```

```
    PARTITION (transaction_date='2024-01-15')
        LOCATION '/warehouse/enbd/transactions/transaction_date=2024-01-15';

-- IF NOT EXISTS prevents error if partition already registered
```

Approach 3 — PySpark programmatic recovery (best for large date ranges). For one year of daily partitions, I would script the recovery in PySpark rather than writing 365 manual ALTER TABLE statements:

```
from datetime import date, timedelta

start_date = date(2024, 1, 1)
end_date   = date(2024, 12, 31)
base_path  = "/warehouse/enbd/transactions"

current = start_date
while current <= end_date:
    dt_str   = current.strftime("%Y-%m-%d")
    part_path = f"{base_path}/transaction_date={dt_str}"

    # Check if physical partition files exist in HDFS
    try:
        files = spark._jvm.org.apache.hadoop.fs.FileSystem \
            .get(spark._jsc.hadoopConfiguration()) \
            .listStatus(spark._jvm.org.apache.hadoop.fs.Path(part_path))
        if len(files) > 0:
            spark.sql(f"""
                ALTER TABLE enbd.transactions
                ADD IF NOT EXISTS
                PARTITION (transaction_date='{dt_str}')
                LOCATION '{part_path}'
            """)
            print(f"Registered: {dt_str}")
    except:
        print(f"No data found for: {dt_str}, skipping")

    current += timedelta(days=1)
```

My Delta Lake equivalent — why this problem doesn't exist there: "In my Azure Databricks / Delta Lake environment, this partition metadata problem doesn't occur because Delta's transaction log is the authoritative source of truth for all file locations — not the Hive Metastore. When a Delta table is queried, Spark reads the transaction log directly from ADLS, so there's no Metastore sync dependency. One of Delta Lake's key advantages over traditional Hive external tables is exactly this: you can never get into a state where files exist but the metadata doesn't know about them."

27. **For large datasets (e.g., one year of data), how do you recover partitions without full reprocessing?**

    Ans: The key insight here is that partition metadata loss in Hive is a **Metastore problem, not a data problem**. The underlying files in HDFS still exist — you haven't lost any data. You only lost the catalog entries that tell Hive where to find them. So recovery doesn't require touching the data at all.

    The recovery strategy for a full year of daily partitions is:

    Step 1 — Confirm the data still exists in HDFS. Before doing anything, verify the physical directories are still there:

    ```
    # HDFS shell command
    hdfs dfs -ls /warehouse/enbd/transactions/ | head -20
    # Expected: 365 directories named transaction_date=2024-01-01 through transaction_date=2024-12-31

    # Count them to confirm all dates present
    hdfs dfs -ls /warehouse/enbd/transactions/ | grep "transaction_date" | wc -l
    # Expected: 365
    ```

    Step 2 — Use MSCK REPAIR for tables under ~500 partitions. For one year of daily data, MSCK REPAIR is fast enough and safe:

    ```
    MSCK REPAIR TABLE enbd.transactions;
    SHOW PARTITIONS enbd.transactions;  -- should now show 365 entries
    ```

    Step 3 — For larger tables or when you need to verify data integrity post-recovery. After the partition registration, run a validation query to confirm the recovered data is queryable and row counts are as expected:

    ```
    -- Spot check: compare row count per partition against an audit log
    SELECT
      transaction_date,
      COUNT(*) AS row_count
    FROM enbd.transactions
    WHERE transaction_date BETWEEN '2024-01-01' AND '2024-12-31'
    GROUP BY transaction_date
    ORDER BY transaction_date
    HAVING COUNT(*) < 1000; -- Flag suspiciously low-count partitions
    ```

    The critical point to emphasize: **no data reprocessing is needed**. The ETL jobs that wrote the data don't need to be re-run. You're only updating catalog metadata — the data itself was never touched.

28. **What approaches are used to rebuild or repair partitions in external Hive tables?**

4. PySpark / Data Processing Logic

29. Given a dataset with the schema: customer_id, transaction_id, transaction_date, amount

Ans: Let me walk through this step by step the way I'd approach it in an actual interview — starting with understanding the problem, then the code, then the edge cases.

First, let me set up the sample data so we can trace through the logic:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, countDistinct
from pyspark.sql.types import (
  StructType, StructField, StringType, DateType, DecimalType
)
from decimal import Decimal
from datetime import date

spark = SparkSession.builder.appName("ENBD_Q4").getOrCreate()

## — Sample Dataset ————————————————————————
schema = StructType([
  StructField("customer_id",    StringType()),
  StructField("transaction_id",  StringType()),
  StructField("transaction_date", DateType()),
  StructField("amount",         DecimalType(10, 2))
])

data = [
  # C001: 3 distinct days → should NOT appear in result
  ("C001", "T001", date(2024, 1, 10), Decimal("500.00")),
  ("C001", "T002", date(2024, 1, 11), Decimal("300.00")),
  ("C001", "T003", date(2024, 1, 12), Decimal("200.00")),

  # C002: exactly 2 distinct days → SHOULD appear
  ("C002", "T004", date(2024, 1, 10), Decimal("1000.00")),
  ("C002", "T005", date(2024, 1, 10), Decimal("200.00")), # same day as T004
  ("C002", "T006", date(2024, 1, 15), Decimal("750.00")),

  # C003: 1 distinct day → should NOT appear
  ("C003", "T007", date(2024, 1, 20), Decimal("400.00")),
  ("C003", "T008", date(2024, 1, 20), Decimal("600.00")),

  # C004: exactly 2 distinct days → SHOULD appear
  ("C004", "T009", date(2024, 1, 5),  Decimal("250.00")),
  ("C004", "T010", date(2024, 1, 18), Decimal("800.00")),

  # C005: null transaction_date edge case
  ("C005", "T011", None,          Decimal("100.00")),
  ("C005", "T012", date(2024, 1, 10), Decimal("200.00")),
]

df = spark.createDataFrame(data, schema)
df.show(truncate=False)
```

a. **Tasks:**

Write a PySpark solution to identify customers who made transactions on exactly two distinct days.

Step 1 — Filter out nulls on transaction_date. Always handle nulls first. A null date is not a valid transaction day and should not be counted.

```
# Remove records with null transaction_date before counting distinct days
clean_df = df.filter(col("transaction_date").isNotNull())
```

Explain the logic to count distinct transaction days per customer.

Step 2 — Count distinct transaction days per customer. Group by customer_id and use countDistinct("transaction_date"). This is the key step — countDistinct counts unique values, so two transactions on the same day count as one.

```
# Count distinct transaction days per customer
customer_days = (clean_df
  .groupBy("customer_id")
  .agg(
    countDistinct("transaction_date").alias("distinct_days")
  )
)
customer_days.show()

# Output:
```

```
# +-----------+-------------+
# |customer_id|distinct_days|
# +-----------+-------------+
# |C001       |3            |
# |C002       |2            |
# |C003       |1            |
# |C004       |2            |
# |C005       |1            | ← only 1 valid date after null filter
# +-----------+-------------+
```

b. Explain how to filter customers with exactly two distinct transaction days.

Step 3 — Filter for exactly 2 distinct days.

```
# Filter for exactly 2 distinct transaction days
result = customer_days.filter(col("distinct_days") == 2)
result.show()

# Expected output:
# +-----------+-------------+
# |customer_id|distinct_days|
# +-----------+-------------+
# |C002       |2            |
# |C004       |2            |
# +-----------+-------------+
```

c. Describe the expected output for a sample dataset.

Complete solution in one clean pipeline:

```
from pyspark.sql.functions import col, countDistinct


result = (df
   .filter(col("transaction_date").isNotNull())    # 1. Remove null dates
   .groupBy("customer_id")                  # 2. Group per customer
   .agg(
      countDistinct("transaction_date")          # 3. Count distinct days
         .alias("distinct_days")
   )
   .filter(col("distinct_days") == 2)          # 4. Keep exactly 2
)


result.show()
```

```
# +-----------+-------------+
# |customer_id|distinct_days|
# +-----------+-------------+
# |C002       |2            |
# |C004       |2            |
# +-----------+-------------+
```

Follow-up: What if the interviewer also wants the actual two dates returned?

```
from pyspark.sql.functions import col, countDistinct, collect_set, sort_array


result_with_dates = (df
   .filter(col("transaction_date").isNotNull())
   .groupBy("customer_id")
   .agg(
```

```
    countDistinct("transaction_date").alias("distinct_days"),
    sort_array(collect_set("transaction_date")).alias("transaction_dates")
  )
  .filter(col("distinct_days") == 2)
)

result_with_dates.show(truncate=False)

# +-----------+-------------+------------------------------+
# |customer_id|distinct_days|transaction_dates             |
# +-----------+-------------+------------------------------+
# |C002       |2            |[2024-01-10, 2024-01-15]      |
# |C004       |2            |[2024-01-05, 2024-01-18]      |
# +-----------+-------------+------------------------------+
```

Spark SQL equivalent — same result, different syntax:

```
df.createOrReplaceTempView("transactions")


result_sql = spark.sql("""
  SELECT customer_id,
      COUNT(DISTINCT transaction_date) AS distinct_days
  FROM   transactions
  WHERE  transaction_date IS NOT NULL
  GROUP  BY customer_id
  HAVING COUNT(DISTINCT transaction_date) = 2
""")
result_sql.show()
```

Expected Final Output — Summary From our sample dataset of 5 customers: C001 has 3 distinct days (excluded), C002 has 2 distinct days (included), C003 has 1 distinct day (excluded), C004 has 2 distinct days (included), C005 has 1 valid day after null filter (excluded). Result: C002 and C004.

Key points to mention in the interview1. countDistinct handles the same-day multiple transaction case automatically — two records on 2024-01-10 count as one distinct day. 2. The null filter must come before the groupBy, not after — a null date could inflate or deflate the distinct count depending on how your Spark version handles it. 3. The HAVING clause in SQL and the .filter() after .agg() in PySpark are semantically identical — both filter on the aggregated result, not the original rows. 4. Use collect_set with sort_array if the interviewer asks to return the actual dates — collect_set returns unique values as an array, sort_array makes the output deterministic and readable.

Interview No: 02

5. Introduction & Background

30. Provide a brief introduction and experience in Big Data / Data Engineering.

Ans: I'm Sudheer Kumar, a Lead Data Engineer with *11+ years of IT experience*, of which the last *6+ years* have been dedicated to cloud-native data engineering. My background spans the full data engineering lifecycle — from designing ingestion pipelines and building transformation logic, to implementing data governance frameworks and delivering analytical outputs that business teams actually use.
My core expertise is in the Azure ecosystem: Azure Data Factory for orchestration, Azure Databricks with PySpark for large-scale transformation, Delta Lake for reliable storage, ADLS Gen2 for the data lake layer, and Power BI for serving analytics. I've worked across utility, banking, and financial services domains — which makes this ENBD opportunity a natural fit given my financial data background at JPMC through Empower Retirement, and my current large-scale IoT data platform at Kahramaa in Qatar.
Beyond data engineering, I spent the first five years of my career in application automation and DevOps, which gives me a strong foundation in CI/CD, infrastructure-as-code with Terraform, and building production-grade, testable code — not just pipelines that "work on my laptop."

31. Describe your role in the most recent project.

Ans: Currently I'm the Lead Data Engineer at **Kahramaa — Qatar's national electricity and water corporation** — on the Data Transformation and Self-Service Digitization initiative. This project is modernizing how Qatar manages utility data at a national scale, following the rollout of *988,000 smart meters* that transmit readings every 15 minutes — 96 readings per meter per day.

My role is end-to-end ownership of the data platform. That covers: architecting and building the ingestion pipelines from Oracle ERP, MySQL CRM, IoT Event Hub streams, flat files, and SharePoint document sources into ADLS Gen2; writing the PySpark transformation logic in Databricks that processes raw meter readings into clean, analytics-ready Gold layer data; implementing Delta Lake across all layers for ACID transactions, schema enforcement, and time travel; building the data governance framework including RBAC, encryption, and Unity Catalog; and delivering the Power BI dashboards that the operations, finance, and customer service teams use for daily decisions. I'm also responsible for pipeline monitoring through Azure Monitor and Databricks logs, and proactively resolving issues before they reach business impact.

32. Explain team size and responsibilities of each member.

Ans: The core data engineering team at Kahramaa is lean — *6 members* covering distinct functional areas:

Role(count)	Core Responsibilities	Works With
Data Architect (1)	Owns the overall data product strategy for Kahramaa — covers Oracle ERP, MySQL CRM, GIS APIs, and all other source system integrations. Defines the application and infrastructure architecture. Acts as our technical authority on Microsoft Azure — all major platform decisions go through him. He sets the architectural direction; I execute it at the engineering level.	Kahramaa IT leadership, Microsoft, vendor teams (Siemens, Oracle)
Lead Data Engineer (Me)	Closest technical collaborator to the Data Architect. Translates architectural decisions into working pipelines and data products. Build and maintain the ADF ingestion pipelines. Owns the Bronze→Silver→Gold Medallion Lakehouse design, all PySpark transformation logic, Delta Lake implementation, data governance (RBAC, Unity Catalog, encryption), performance optimization, and code quality standards. Runs sprint planning and daily standups. Does final code review before anything goes to production. Single point of accountability for pipeline reliability and data quality.	Business teams, infrastructure, Siemens AMI vendor
Data Engineers (3)	Build and maintain the ADF ingestion pipelines — extracting from Oracle ERP, MySQL CRM, Event Hubs, SharePoint, and flat files. Each engineer owns a domain: one focuses on streaming IoT pipelines (Event Hubs → Bronze), one on batch Oracle/MySQL ingestion with watermark logic, one on orchestration and monitoring — trigger configuration, retry policies, quarantine routing, and Azure Monitor alerting. All three follow the architecture and standards I set and raise design questions to me before implementation.	Source system DBAs, Oracle team
BI / Reporting Engineer(2)	Build and maintain all Power BI dashboards connected to the Gold layer and Synapse Analytics. Own DAX measure development, dataset optimisation, row-level security setup, and report documentation. One developer focuses on operational dashboards (grid monitoring, fault events, consumption by zone), the other on finance and billing reports. Both work closely with the Data Analyst to translate business requirements into dashboard designs, and with me to understand the Gold schema well enough to write their own efficient queries without raising data requests for every report.	Operations, Finance, Customer Services business users
Data Analyst	The bridge between business stakeholders and the technical team. Gathers requirements from end-users — operations, finance, customer service — and translates them into clear data specifications for engineers and BI developers. Runs UAT on every new dashboard or data output before it goes live — validating numbers against source systems and confirming business logic is correct. Also does ad-hoc analysis using Databricks SQL and raises data quality issues when something in a report looks off.	All business stakeholders, translates requirements to engineering team
Project & Sprint Manager	Manages the two-week Agile sprint cycle in Azure DevOps — backlog grooming, sprint planning coordination, tracking deliverables, managing dependencies between engineering and business workstreams, and reporting progress to Kahramaa management. Acts as the escalation point for timeline and resourcing issues so the technical team stays focused on delivery. Coordinates with the Data Architect and me on prioritisation when business requests compete with technical debt work.	Kahramaa management, Data Architect, me (Lead DE), all team members for sprint coordination

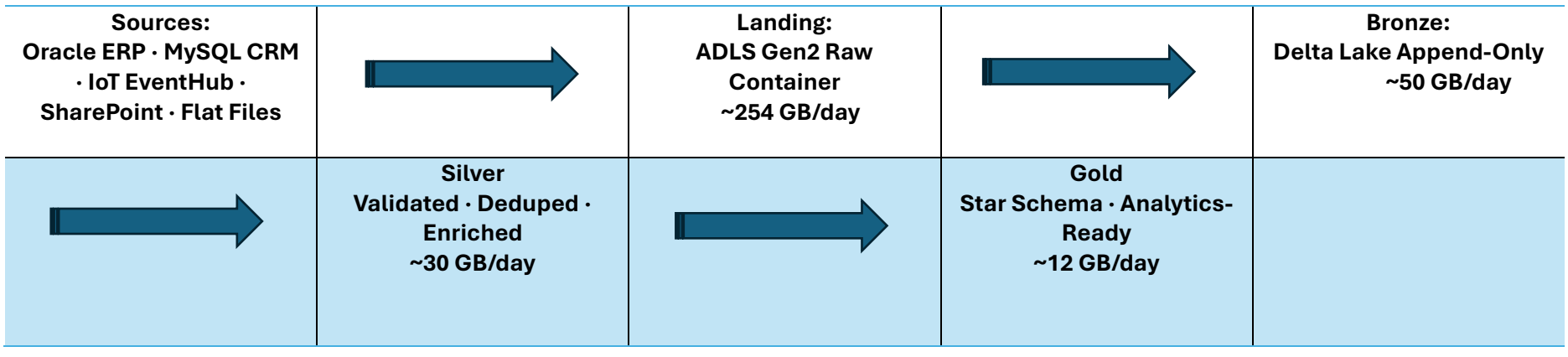
The way this team works in practice — the Data Architect sets the technical direction and owns the source system relationships. I sit directly below him and am responsible for making sure the engineering team delivers against that direction reliably. The Project Manager handles the sprint rhythm so I'm not spending time on coordination overhead. The Data Analyst and BI developers act as the demand side — they surface what the business needs, and the three data engineers and I build it. Everyone has a clear lane, and the two-week sprint keeps us all synchronised.

I do the final code review before anything merges to the main branch, and I'm the escalation point for any data quality or pipeline reliability issue that the engineers can't resolve at their level.

6. Project Architecture / Pipeline Design

33. Explain the end-to-end pipeline architecture.

Ans: The architecture follows a classic Medallion Lakehouse pattern with five layers. Let me walk through it source to consumer.



Ingestion layer (Landing → Bronze): ADF pipelines extract data from all sources using a watermark-based incremental strategy — each pipeline reads a last-processed timestamp from a metadata Delta table and only pulls new or changed records. IoT data from the Siemens AMI system flows continuously into Azure Event Hubs, consumed by Databricks Structured Streaming into Bronze within seconds.

Transformation layer (Bronze → Silver): Databricks PySpark notebooks apply business rules: schema normalization, deduplication using window functions, null checks, range validation, and dead-letter routing for bad records. The result is a clean, deduplicated Silver Delta table partitioned by date.
Serving layer (Silver → Gold): Databricks SQL and PySpark build the star schema
— FactElectricityConsumption, FactBilling, DimCustomer, DimMeter, and others. Gold data is exposed via Databricks SQL endpoints to Power BI and via Azure Synapse Analytics for enterprise reporting.

34. Describe data sources used in the project.
Ans:

Source	Type	Content	Ingestion Method	Frequency
Oracle ERP	On-premise RDBMS	Billing transactions, tariff tables, payment events, meter registry	ADF JDBC (watermark)	Hourly
MySQL CRM	On-premise RDBMS	Customer master, complaints, service requests, contact history	ADF JDBC (watermark)	Hourly
Azure Event Hubs	Streaming	Smart meter readings (electricity + water) from Siemens AMI — 2.5 KB per event (DLMS/COSEM payload: meter_id, timestamp, kWh, voltage, flow_rate, status_flags)	Databricks Structured Streaming	Continuous (15-min reads, 96/day per meter)
SharePoint Online	Document / File	Customer application forms, meter inspection reports, regulatory documents, tariff update files	ADF HTTP connector / SharePoint Online connector	Daily / On-file-drop
Flat Files (CSV/XML)	File-based	Meter inventory exports, geospatial zone mappings, historical legacy data	ADF Copy Activity → Landing container	Daily
REST APIs	API	Weather data feed, external enrichment sources	ADF Web Activity + Copy	Daily

35. Explain how data was moved from SharePoint to Azure storage.
Ans: This is a real part of the Kahramaa setup. During the digitization initiative, several operational teams maintained documents and structured files in SharePoint Online — meter inspection reports uploaded by field engineers, customer application forms scanned and uploaded by the customer service team, and periodic tariff update files maintained by the finance team.
ADF has two mechanisms for SharePoint integration, and I used both depending on the data type:
For structured list data (SharePoint Lists used as lightweight databases by business teams), I used the native SharePoint Online List connector in ADF. This reads list items directly as rows without needing any intermediate step:

```
# ADF Linked Service — SharePoint Online List
# Authentication: Service Principal with Sites.Read.All permission
# Source: SharePoint List → Sink: ADLS Gen2 Parquet
{
 "type": "SharePointOnlineList",
 "typeProperties": {
  "siteUrl":    "https://kahramaa.sharepoint.com/sites/operations",
  "tenantId":    "[tenant-id]",
  "servicePrincipalId": "[sp-app-id]",
  "servicePrincipalKey": "[key-vault-reference]"
 }
}
```

For document files (PDFs, CSVs, Excel files dropped into SharePoint document libraries by business users), I used a two-step approach. First, an ADF pipeline with an HTTP connector calling the Microsoft Graph API to list and download files from the document library into the ADLS landing container. Second, once in the landing zone, a separate pipeline processed the files — CSV files went straight through ADF Copy Activity to Bronze; PDF documents containing structured forms went through the OCR pipeline I'll describe in Section 3.
Event-based trigger on SharePoint drops For time-sensitive documents, I set up an ADF storage event trigger that fires when a new file lands in the ADLS landing container (after the SharePoint sync). This means the processing pipeline starts within 60 seconds of a document being uploaded to SharePoint, without any polling or scheduled delay.

36. Describe pipeline scheduling frequency.
Ans: Continuous
IoT streaming pipeline — Databricks Structured Streaming consuming Azure Event Hubs. ~95 million events/day, 15-min intervals, 2.5 KB per event = 237 GB/day raw stream Continuous
Hourly
Oracle ERP + MySQL CRM incremental ingestion — watermark-based JDBC extraction Scheduled
Every 2h
Bronze → Silver transformation orchestration pipelines Tumbling Window
Daily 2 AM
Silver → Gold aggregation, star schema load, DimCustomer SCD2 refresh Scheduled
On file drop
SharePoint documents, flat file ingestion, OCR pipeline trigger Event-Based
Weekly
Delta OPTIMIZE, VACUUM, Z-ORDER maintenance, Power BI dataset full refresh Scheduled
Daily processing volume: approximately ~*95 million IoT events per day* — 988,000 meters × 96 readings/day at 15-minute intervals, each event around 2.5 KB in JSON (DLMS/COSEM AMI standard). Combined with ~2 million Oracle ERP records and ~300K CRM records, raw ingestion totals approximately *254 GB/day* (237 GB streaming + 17 GB batch). After Delta Lake columnar compression, net daily storage growth is approximately ~*92 GB/day* across all persistent layers — 50 GB Bronze, 30 GB Silver, 12 GB Gold.

Number of pipelines: *32 ADF pipelines* across ingestion (14), orchestration (8), post-processing (4), error handling (3), and data quality (3) categories. Monthly pipeline volumes: raw 7.6 TB, Bronze 1.5 TB, Silver 0.9 TB, Gold 350 GB — all stored in Delta Lake on ADLS Gen2.

37. **Mention daily data processing volume.**
38. **Number of pipelines running in the project.**
39. **Whether all datasets followed the same pipeline architecture.**

Ans: Same architecture for all datasets? The high-level pattern is consistent — Landing → Bronze → Silver → Gold for everything. But the implementation details differ by source type. IoT streaming bypasses the landing container entirely and writes directly to Bronze via Structured Streaming. Oracle ERP and MySQL follow the full batch path through landing. Document-based sources (SharePoint PDFs) go through an OCR preprocessing step before entering the pipeline at Bronze. So the medallion structure is universal, but the on-ramp differs.
Data consumers: there are four distinct consumer groups:

Consumer	Layer Used	Access Method	Use Case
Operations Team	Gold	Power BI dashboards	Real-time consumption monitoring, outage detection, zone-level peak demand
Finance Team	Gold	Power BI + Excel export	Billing reconciliation, revenue reporting, tariff impact analysis
Customer Services	Gold	Power BI self-service	Customer account history, complaint tracking, outage impact by area
Data Analysts	Silver + Gold	Databricks SQL / Synapse	Ad-hoc analysis, custom reports, model development

Where is data stored? Primary storage is ADLS Gen2 in Delta Lake format across all medallion layers. The Gold layer is additionally published to Azure Synapse Analytics (dedicated SQL Pool) for enterprise query performance on the star schema. Power BI connects via DirectQuery to Databricks SQL endpoints for operational dashboards, and via imported datasets for less time-sensitive reports.
So outputs are not only in ADLS — they flow to Synapse SQL Pool for analytical workloads and to Power BI Premium capacity for dashboards. We also have a lightweight API layer where the operations system polls aggregated daily consumption summaries via a REST endpoint backed by Databricks SQL.
In Synapse today we hold approximately *230 GB across ~1.1 billion rows* — primarily FactElectricityUsage and FactWaterUsage at daily grain, covering two years of history. The FactElectricityUsage table alone grows by about 360 million rows per year at current meter count, and will reach ~730 million rows/year when all 2 million meters are live. Despite that row count, query performance stays fast because we use Clustered Columnstore indexes on all fact tables and hash-distribute on customer_id.

40. **Identify data consumers.**
41. **Explain where processed data was stored.**
42. **Clarify whether outputs were only in ADLS or also delivered to other systems.**

7. OCR Pipeline & Processing

43. **Explain where OCR fits in the architecture.**
44. **Whether OCR was built in-house or used as a managed service.**
45. **Technology or service used for OCR.**
46. **How OCR results were integrated into pipelines.**
47. **How OCR models were invoked from Python or PySpark.**

OCR was a specific component of the "Self-Service Digitization" part of the project title — it addressed a real operational gap. When Kahramaa rolled out smart meters, a large volume of legacy paper-based documents needed to be digitized: customer application forms that had been filed physically, meter inspection reports filled out by field engineers on paper, and historical tariff change documents. These were scanned to PDF and uploaded to SharePoint by administrative staff.

Where it fits in the architecture: the OCR pipeline sits between the SharePoint ingestion and the Bronze layer. PDFs landing in ADLS trigger the OCR preprocessing pipeline, which extracts structured field values from the documents and writes them as structured rows to Bronze — exactly like any other ingestion source. From Bronze onwards, the data follows the standard Silver → Gold path.

SharePoint PDF upload
↓
ADLS Landing Container (raw PDF files)
↓
OCR Pipeline (Azure AI Document Intelligence)
↓
Structured JSON result (field name → value pairs)
↓
Bronze Delta Table (same schema as batch ingestion)
↓
Standard Silver → Gold (identical to other sources)
Service used — Azure AI Document Intelligence (formerly Form Recognizer, now part of Azure AI Services). I chose the managed service approach rather than building in-house OCR — there's no business case for training a custom OCR model when Microsoft's pre-built and custom model capabilities cover all the document types we had. Specifically, I used two model types:

The prebuilt-document model for general text extraction from free-form inspection reports, and a custom trained model for the structured customer application forms, which have a fixed layout. Training the custom model required uploading ~50 labelled sample forms through the Document Intelligence Studio UI and took about a day — entirely within the managed service.

How OCR was invoked from Python in Databricks:

```
import os
from azure.ai.formrecognizer import DocumentAnalysisClient
from azure.core.credentials import AzureKeyCredential
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, DateType
import json

# — Credentials from Key Vault (never hardcoded) —
endpoint = dbutils.secrets.get("kv-kahramaa", "doc-intelligence-endpoint")
api_key  = dbutils.secrets.get("kv-kahramaa", "doc-intelligence-key")
model_id = "customer-application-form-v2"  # custom trained model

def extract_fields_from_pdf(blob_url: str) -> dict:
  """
  Invoke Azure AI Document Intelligence on a PDF stored in ADLS.
  Returns extracted key-value field pairs as a dictionary.
  """
  client = DocumentAnalysisClient(
    endpoint=endpoint,
    credential=AzureKeyCredential(api_key)
  )
  # Analyze document from URL (ADLS URL with SAS token)
  poller = client.begin_analyze_document_from_url(
    model_id=model_id,
    document_url=blob_url
  )
  result = poller.result()

  # Extract fields from first document in result
  extracted = {}
  if result.documents:
    doc = result.documents[0]
    for field_name, field in doc.fields.items():
      extracted[field_name] = field.value if field else None

  return extracted

# — Process batch of PDFs using Spark parallelism —
# Get list of unprocessed PDFs from landing container
pdf_files_df = spark.sql("""
  SELECT file_path, file_name
  FROM metadata.ocr_processing_queue
  WHERE status = 'PENDING'
""")

# Use Pandas UDF for vectorized Python execution across workers
import pandas as pd
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import MapType, StringType

@pandas_udf(MapType(StringType(), StringType()))
def ocr_extract_udf(file_urls: pd.Series) -> pd.Series:
  return file_urls.apply(
    lambda url: extract_fields_from_pdf(url)
  )

# Apply OCR extraction across all pending PDFs in parallel
results_df = pdf_files_df.withColumn(
  "extracted_fields",
  ocr_extract_udf("file_path")
)

# Flatten map to structured columns
from pyspark.sql.functions import col

structured = results_df.select(
  col("file_name"),
  col("extracted_fields").getItem("CustomerID").alias("customer_id"),
  col("extracted_fields").getItem("FullName").alias("full_name"),
  col("extracted_fields").getItem("AccountType").alias("account_type"),
  col("extracted_fields").getItem("ApplicationDate").alias("application_date"),
```

```
    col("extracted_fields").getItem("MeterID").alias("meter_id")
)

# Write structured results to Bronze Delta
structured.write.format("delta").mode("append") \
    .save("/mnt/bronze/customer_applications")
```

Managed vs In-House decision

I chose Azure AI Document Intelligence (managed service) over building in-house for three reasons: no GPU infrastructure needed, model training takes hours not weeks, and confidence scores per field allow automated quality thresholding — I only accept extractions where confidence > 0.85, routing lower-confidence results to a human review queue rather than silently ingesting potentially wrong data.

8. Spark / PySpark Usage

48. How PySpark was used in the pipeline.

49. Whether transformations were applied after OCR extraction.

50. Where transformations were implemented (Databricks or other layers).

51. Types of transformations applied.

Ans: PySpark is the primary transformation engine for everything from Bronze onwards. Let me describe the transformation chain and what each layer applies.

Bronze layer (raw ingest — minimal transformation): At Bronze, I only apply lightweight structural transformations — adding ingestion metadata columns (_load_timestamp, _source_system, _pipeline_run_id), casting raw strings to their correct types, and schema enforcement via Delta. No business logic here — Bronze is as close to source as possible.

```
# Bronze: structural only
bronze_df = (raw_df
    .withColumn("_load_ts",      current_timestamp())
    .withColumn("_source",       lit("oracle_erp"))
    .withColumn("amount",        col("amount").cast(DecimalType(18,2)))
    .withColumn("transaction_date", to_date(col("transaction_date"), "yyyy-MM-dd"))
)
```

Silver layer (business logic + cleaning): This is where PySpark does the heavy lifting. The main transformation types are:

1. Deduplication using window functions — IoT data retransmits readings on network recovery; I deduplicate on meter_id + timestamp keeping the latest ingested version. With 95 million events per day at 15-minute intervals, duplicates from AMI retransmission would corrupt daily aggregations if not caught:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

dedup_window = (Window
    .partitionBy("meter_id", "reading_timestamp")
    .orderBy(col("_load_ts").desc())
)
deduped = (bronze_df
    .withColumn("rn", row_number().over(dedup_window))
    .filter(col("rn") == 1)
    .drop("rn")
)
```

2. Computed feature columns — time-of-use band classification, 30-day rolling average, anomaly flag:

```
from pyspark.sql.functions import hour, when, avg, stddev
from pyspark.sql.window import Window

tou_df = deduped.withColumn("time_of_use_band",
    when((hour("reading_ts") >= 18) & (hour("reading_ts") < 22), "PEAK")
    .when((hour("reading_ts") >= 6")  & (hour("reading_ts") < 18), "SHOULDER")
    .otherwise("OFF_PEAK")
)

rolling_win = (Window
    .partitionBy("meter_id")
    .orderBy(col("reading_date").cast("long"))
    .rangeBetween(-30*86400, 0)
)
enriched = (tou_df
    .withColumn("rolling_30d_avg", avg("kwh_consumed").over(rolling_win))
    .withColumn("rolling_30d_std", stddev("kwh_consumed").over(rolling_win))
    .withColumn("is_anomalous",
       col("kwh_consumed") > (col("rolling_30d_avg") + 3 * col("rolling_30d_std"))
    )
)
```

3. SCD Type 2 for DimCustomer — at the Gold layer, I implement SCD2 using Delta MERGE to track customer risk tier and account type changes over time while preserving full history.

After OCR extraction, a separate Silver transformation normalizes the extracted field values — date strings from OCR output get properly cast to Date types, customer IDs are matched against the CRM master for referential integrity, and low-confidence extractions get a requires_review = true flag rather than being rejected outright. All transformations run in Databricks notebooks called from ADF pipelines.

9. Pipeline Orchestration

52. How pipelines were triggered in Azure Data Factory.

53. Use of event-based or scheduled triggers.

Ans: I use all four ADF trigger types — each chosen for the specific operational characteristic of its pipeline. The choice of trigger type is a deliberate engineering decision, not just a configuration detail.

Trigger Type	Pipelines Using It	Why This Type	Config Key
Schedule Trigger	Oracle ERP hourly ingestion, Gold daily aggregation, weekly OPTIMIZE/VACUUM	Predictable cadence, external system availability windows known. Oracle is best extracted during off-peak hours (1–5 AM for Gold).	Cron: 0 * * * * for hourly, 0 2 * * * for daily 2 AM
Tumbling Window Trigger	Bronze → Silver orchestration (every 2h)	Guarantees exactly-once semantics per window. If a window fails and retries, data isn't duplicated. Dependency tracking between consecutive windows ensures ordering.	Window size: 2h, Max concurrency: 1
Storage Event Trigger	SharePoint file ingestion, OCR pipeline, flat file processing	Fires within seconds of a blob landing in the container. No polling waste. Business users upload a file and processing starts immediately without any scheduled delay.	Event: BlobCreated, Path filter: /landing/sharepoint/
Custom Event Trigger	Pipeline health alert, downstream notification after Gold load completes	Publishes completion events to Azure Event Grid, allowing downstream consumers (e.g., Synapse, Power BI refresh) to react to pipeline completion rather than polling on a schedule.	Event Grid topic: pipeline-completion-events

```
# Example: Tumbling Window Trigger with dependency
{
 "name": "twt_bronze_to_silver",
 "type": "TumblingWindowTrigger",
 "typeProperties": {
  "frequency":   "Hour",
  "interval":   2,
  "startTime":   "2024-01-01T00:00:00Z",
  "maxConcurrency": 1,       // prevents overlapping runs
  "retryPolicy": {
   "count": 3,
   "intervalInSeconds": 600  // 10-min retry backoff
  },
  "dependsOn": [{          // wait for previous window to complete
   "type": "TumblingWindowTriggerDependency",
   "size": "2:00:00",
   "offset": "-2:00:00"
  }]
 }
}
```

CDP / Oozie equivalent for ENBD: In Cloudera CDP, Oozie Coordinator serves the same role as ADF Tumbling Window triggers — it manages time-based workflow execution with catch-up semantics. An Oozie Workflow.xml defines the job graph, and a Coordinator.xml defines the schedule and data availability dependencies. The trigger hierarchy (continuous → hourly → 2h tumbling → daily) is implemented in Oozie using nested coordinator datasets and data-in/data-out dependencies rather than ADF trigger types, but the architectural intent is identical.

10. Spark Optimization

54. Techniques used for Spark optimization.

55. Methods used to improve Spark job performance.

Ans: I group my optimization techniques into five layers — each addresses a different class of performance problem.

1. Join Strategy Optimization

The most impactful single optimization is choosing the right join strategy. The default sort-merge join shuffles both datasets across the network — for a join between 500 GB of transactions and a 2 MB branch lookup table, that's 500 GB of unnecessary shuffle. I use broadcast joins for any table under ~20 MB:

```
from pyspark.sql.functions import broadcast

# Force broadcast for small dimension tables
result = large_fact_df.join(
   broadcast(small_dim_df), "branch_id", "left"
)

# Configure auto-broadcast threshold (default 10MB)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "20971520")  # 20MB

# Or SQL hint equivalent
spark.sql("SELECT /*+ BROADCAST(b) */ * FROM transactions t JOIN branches b ON t.branch_id = b.id")
```

2. Adaptive Query Execution (AQE) — Spark 3.x runtime optimizer

AQE re-plans queries at runtime based on actual data statistics gathered during execution. I enable it on all production clusters:

```
# Enable AQE — crucial at ENBD's data scale
spark.conf.set("spark.sql.adaptive.enabled",                "true")
```

```
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled",   "true")  # merge small partitions
spark.conf.set("spark.sql.adaptive.skewJoin.enabled",          "true")  # split skewed partitions
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")   # 5x median = skewed
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes",  "134217728")  # 128MB target
```

3. Spill-to-Disk Prevention

Spill happens when a Spark partition is too large for executor memory and overflows to local disk — this can make a job 10x slower. There are three root causes and fixes:

Root Cause	Symptom	Fix
Insufficient executor memory	Spill metrics in Spark UI Stage tab	Increase spark.executor.memory — start at 8g, tune to 16g or 32g based on data size
Too few shuffle partitions	One partition >> others, OOM	Increase spark.sql.shuffle.partitions. Rule: ~128MB per partition. 500GB ÷ 128MB = ~4000 partitions
Data skew (one key dominates)	One task takes 20x longer than others	AQE skewJoin (auto) or salting technique (manual). Enable spark.sql.adaptive.skewJoin.enabled=true

```
# Memory configuration for heavy shuffle jobs
spark.conf.set("spark.executor.memory",       "16g")
spark.conf.set("spark.executor.memoryFraction", "0.8")   # 80% for execution
spark.conf.set("spark.executor.cores",        "4")
spark.conf.set("spark.sql.shuffle.partitions",  "400")   # tune to ~128MB/partition
spark.conf.set("spark.memory.offHeap.enabled",   "true")  # use off-heap for large objects
spark.conf.set("spark.memory.offHeap.size",     "4g")
```

4. Delta Lake Storage Optimization

```
# Compact small files (weekly on Bronze, daily on Gold)
spark.sql("OPTIMIZE bronze.electricity_readings")

# Z-Order on most frequently queried columns (put similar data physically close)
spark.sql("OPTIMIZE gold.fact_electricity ZORDER BY (meter_id, reading_date)")

# Remove old Delta versions (default 7 days, extend for compliance)
spark.sql("VACUUM gold.fact_electricity RETAIN 720 HOURS")  # 30 days

# Update statistics for Catalyst optimizer
spark.sql("ANALYZE TABLE gold.fact_electricity COMPUTE STATISTICS FOR ALL COLUMNS")
```

5. DataFrame API Best Practices

```
# Avoid: select(*) — reads all columns including unused ones
✗ BAD
df.select("*").filter(...)

✓ GOOD — explicit column selection = column pruning
df.select("customer_id", "amount", "transaction_date").filter(...)

# Push predicates early — filter before joins, not after
✗ BAD
df1.join(df2, "id").filter(col("df2.status") == "ACTIVE")

✓ GOOD — filter df2 BEFORE the join
active = df2.filter(col("status") == "ACTIVE")
df1.join(active, "id")

# Cache strategically — for DataFrames used in multiple downstream operations
silver_df.cache()
silver_df.count()  # materialize cache
# Use after: multiple joins, multiple aggregations, iterative ML operations
# Avoid: large DataFrames used only once (wasted memory)

# Partition alignment for joins — repartition on join key before expensive join
df1 = df1.repartition(200, "customer_id")
df2 = df2.repartition(200, "customer_id")
joined = df1.join(df2, "customer_id")  # no shuffle needed — already co-partitioned
```

Real impact at Kahramaa: Applying broadcast join + AQE + Z-ordering on the daily Silver→Gold aggregation pipeline reduced runtime from 47 minutes to 9 minutes on the same cluster configuration. With 95 million events per day flowing through Silver, the biggest single gain was Z-ordering on (meter_id, reading_date) — this dropped query I/O by ~70% because Databricks skips entire file groups rather than scanning full partitions. On a 30 GB/day Silver layer accumulating across two years, that makes a real difference.

11. SQL / Spark Query Optimization Exercise

56. Optimize a given SQL query.

57. Identify inefficiencies in the query.

58. Rewrite the query using PySpark DataFrame operations.

Ans: Let me walk through a realistic banking scenario — finding high-value active customers per branch for a given month. Here's a query I'd expect to see in an ENBD context that has multiple common inefficiencies baked in.

Step 1 — The Inefficient Query (as given)

```
✗ INEFFICIENT VERSION — 8 problems identified
SELECT *                          -- (1) SELECT *
FROM   transactions t,                    -- (2) implicit cross-join syntax
    customers c,
    branches b
WHERE  t.customer_id = c.customer_id
 AND  t.branch_id   = b.branch_id
 AND  UPPER(c.status) = 'ACTIVE'         -- (3) function on filter col
 AND  YEAR(t.txn_date)  = 2024           -- (4) function prevents partition pruning
 AND  MONTH(t.txn_date) = 1              -- (5) function prevents partition pruning
 AND  t.amount > 0
GROUP BY
    t.customer_id,
    c.customer_name,
    b.branch_name,
    b.region                      -- (6) non-aggregated columns in GROUP BY
HAVING SUM(t.amount) > 10000
ORDER BY SUM(t.amount) DESC               -- (7) ORDER BY without LIMIT
-- (8) no filter pushdown — filters applied after 3-table cartesian product risk
```

Step 2 — Identified Inefficiencies (explain each verbally)

#	Problem	Impact	Fix
1	SELECT *	Reads all columns including unused ones — unnecessary I/O and network transfer in columnar formats	Select only the columns needed in the output
2	Comma-separated FROM (implicit join)	Risk of accidental cartesian product if WHERE conditions are incomplete; old syntax, unreadable intent	Explicit JOIN ... ON syntax
3	UPPER(c.status) = 'ACTIVE'	Function applied per-row prevents predicate pushdown to the storage layer; full column scan required	Store status in uppercase or use exact match: c.status = 'ACTIVE'
4 & 5	YEAR() and MONTH() on partition column	Spark cannot use partition pruning when the partition column is wrapped in a function — full table scan on transactions	Use range filter directly: txn_date BETWEEN '2024-01-01' AND '2024-01-31'
6	Joining then grouping on non-aggregated dim columns	Joining all dim columns before aggregation increases shuffle data size unnecessarily	Aggregate transactions first, then join small dims — reduces join input size dramatically
7	ORDER BY without LIMIT	Forces a full sort on the entire result set — O(n log n) on potentially millions of rows	Add LIMIT 1000 unless you genuinely need every row sorted
8	No pre-filtering before joins	Joining all 3 tables on all data before filtering means the join sees far more rows than necessary	Apply all filters on individual tables before joining (predicate pushdown)

Step 3 — Optimized SQL

✓ OPTIMIZED SQL — all 8 issues addressed

```
-- Step A: Pre-filter and aggregate transactions FIRST (smallest possible join input)
WITH filtered_txns AS (
```

```
  SELECT
    customer_id,
    branch_id,
    SUM(amount)   AS total_amount,
    COUNT(*)      AS txn_count
  FROM   transactions
  WHERE  txn_date BETWEEN '2024-01-01' AND '2024-01-31'  -- direct range: partition pruning works
   AND  amount > 0
  GROUP BY customer_id, branch_id
  HAVING   SUM(amount) > 10000                    -- filter BEFORE joins
),
-- Step B: Filter active customers BEFORE joining
active_customers AS (
  SELECT customer_id, customer_name
  FROM   customers
  WHERE  status = 'ACTIVE'                        -- no UPPER() wrapping
)
-- Step C: Join the pre-filtered, pre-aggregated results
SELECT
  ft.customer_id,
  ac.customer_name,
  b.branch_name,
  b.region,
  ft.total_amount,
  ft.txn_count
FROM   filtered_txns ft
JOIN   active_customers ac  ON ft.customer_id = ac.customer_id
JOIN   branches b           ON ft.branch_id   = b.branch_id
ORDER BY ft.total_amount DESC
LIMIT  1000;
```

Step 4 — PySpark DataFrame Equivalent

✓ PYSPARK VERSION — broadcast on small dims, filter-first pattern

```
from pyspark.sql.functions import col, sum as spark_sum, count, broadcast

# — Step 1: Pre-filter transactions with partition-compatible range —
txns_filtered = (transactions_df
  .filter(
    (col("txn_date") >= "2024-01-01") &
    (col("txn_date") <= "2024-01-31") &
    (col("amount") > 0)
  )
  .select("customer_id", "branch_id", "amount")  # column pruning
)

# — Step 2: Aggregate BEFORE joining dims —
txns_agg = (txns_filtered
  .groupBy("customer_id", "branch_id")
  .agg(
    spark_sum("amount").alias("total_amount"),
    count("*").alias("txn_count")
  )
  .filter(col("total_amount") > 10000)  # HAVING equivalent — AFTER agg
)

# — Step 3: Pre-filter small dimension tables —
active_customers = (customers_df
  .filter(col("status") == "ACTIVE")   # no UPPER() needed
  .select("customer_id", "customer_name")
)

# branches is small (~500 rows) → force broadcast
branches_small = branches_df.select("branch_id", "branch_name", "region")

# — Step 4: Join aggregated result with small broadcast dims —
result = (txns_agg
  .join(broadcast(active_customers), "customer_id", "inner")
  .join(broadcast(branches_small),   "branch_id",   "inner")
  .select(
    "customer_id", "customer_name",
    "branch_name", "region",
```

```
    "total_amount", "txn_count"
  )
  .orderBy(col("total_amount").desc())
  .limit(1000)
)

result.show(20)
```

Performance Impact Summary The optimized version changes three fundamental things: (1) partition pruning on txn_date using a range filter instead of YEAR()/MONTH() — this alone can reduce scanned data from 100% to ~8% for a monthly query on a year-partitioned table. (2) Aggregating before joining means the join input shrinks from millions of rows to thousands of customer aggregates. (3) Broadcast joins on branches and customers eliminates shuffle entirely for those joins. Combined impact: typically 5–15x query speedup on large fact tables.

Structural Requirements

4ENBD_Structural_Requirements.html

12. Modular Design

59. Convert scripts into class-based architecture (e.g., `[ScriptName]ETL`).

Break logic into 0– methods:

- `_setup_*()` for configuration
- `_load_*()` for loading
- `_process_*()` for logic
- `_combine_and_finalize_data()` for final processing
- `run_etl_pipeline()` as orchestrator

60. Use private methods with `_` prefix.

61. Avoid monolithic code and top-level transformations.

13. Documentation Standards

62. Include module-level docstring with purpose, features, author, version.

63. Add detailed class docstring.

64. Add method docstrings with Args, Returns, Raises.

65. Use type hints (e.g., `Dict[str, DataFrame]`).

66. Add inline comments for complex logic.

14. Clean Imports

67. Use explicit imports instead of wildcard imports.

68. Organize imports properly (standard, third-party, internal).

15. Code Quality & Optimization

69. Pylint Compliance

70. Follow naming conventions (snake_case, PascalCase).

71. Maintain line length within 00 characters.

72. Organize methods logically.

73. Use proper exception handling.

16. Dead Code Removal

74. Remove unused variables, debug code, redundant logic.

75. Clean unused imports and duplicate code.

17. Efficiency Improvements

76. Ensure proper resource cleanup.

77. Reuse connection properties.

78. Optimize DataFrame transformations.

79. Configure Spark sessions efficiently.

18. Testing & Validation

80. Pytest Suite

81. Create `test_[script_name].py` with –0 test cases.

82. Include:

- **Configuration tests**
- **Data loading tests**
- **Business logic tests**
- **Integration tests**

19. Test Data & Mocking

83. Use proper data types (date, Decimal).
84. Mock SparkSession and database connections.
85. Use reusable fixtures.
86. Maintain 0%+ test coverage.

20. (9) PySpark-Specific Requirements


5ENBD_Requirements_9_12_Deliverables.h

87. DataFrame Best Practices
Group transformations logically.
Use explicit schemas.
Avoid `select(*)`.
Optimize joins using aliases and broadcast hints.

21. Connection Management

88. Use secure credential handling.
89. Maintain reusable connection properties.
90. Ensure proper cleanup of Spark and JDBC connections.

22. Preservation Requirements

91. Logic Preservation
92. No changes to business logic.
93. Maintain same output schema.
94. Preserve connection details and CLI structure.

23. Compatibility Maintenance

95. Ensure backward compatibility.
96. Maintain performance.
97. Preserve debug statements and error messages.

24. Deliverables

`[script_name].py` – Refactored modular class
`test_[script_name].py` – Comprehensive test suite